

Hangman Pseudocode

The Big Picture First

We're building a Hangman-playing program. It works in three phases:

1. **Startup** — read the dictionary file into memory, organized smartly
2. **Guess a letter** — look at what's still possible, pick the best letter
3. **Process feedback** — cross off words that are now impossible

That's it. Everything below is just the details of how to do each phase efficiently.

Part 1: The Data We Keep Around

Before writing any code, you need to understand what information the program tracks.

Constants (never change)

```
MAX_LEN = 30
// The longest word we'll bother with. Drop anything longer.
// Why: they're rare, and dropping them saves memory.

FREQ_ORDER = "esiarntolcdupmghbyfvkwzxqj"
// Letters ordered by how common they are in dictionary words.
// 'e' appears in the most words, 'j' in the fewest.
// We use this to break ties and as a last-resort fallback.
```

Things we set up ONCE at startup and never change

```
wordData      -- a giant byte[] with every dictionary word jammed together
                Example: "applebananacherry..."

wordOffset    -- an int[] telling us where each word starts in wordData
                Example: [0, 5, 11, 17, ...]
                Word #0 is at positions 0-4 ("apple")
                Word #1 is at positions 5-10 ("banana")
                Word #2 is at positions 11-16 ("cherry")

indicesByLength -- for each length L, the list of word numbers of length L
                Example: indicesByLength[5] = [0, 47, 203, ...]
                        meaning words #0, #47, #203 are all 5 letters long

firstGuess    -- for each length L, the best opening letter
                Example: firstGuess[5] = 's', firstGuess[9] = 'e'
```

Why structure it this way? Because a Java `String` object is expensive (~38 bytes of bookkeeping each), and we have 235,000 of them. Storing words as raw bytes in one big array with an index is way cheaper.

Things that change during a game

```
candidates    -- an int[] of word numbers still possible as the hidden word
candCount     -- how many entries of candidates[] are currently alive
guessed[26]   -- guessed['a'..'z'] = true if we've guessed that letter this game
firstGuessOfWord -- true on the very first guess of a new word
letterCounts[26] -- scratch space for counting letters (declared once, reused)
seen[26]      -- scratch space: while counting a word, which letters have
                we already counted from it? (so "banana" contributes 1 to 'a',
                not 3). Declared once, reused.
```

Part 2: Startup (the Constructor)

What it does: Reads the dictionary file and sets up everything above.

Why it's done in two passes: We want arrays sized exactly right (no wasted space, no resizing). So we first scan the file to *count* things, then scan again to *fill* the arrays.

Pass 1: Count things

```
FUNCTION constructor(wordFile):  
  
    // We want to know:  
    //   - how many total words will we keep? (for wordOffset's size)  
    //   - how many bytes total?                (for wordData's size)  
    //   - how many words at each length?      (for indicesByLength sizes)  
  
    countPerLen := [0, 0, 0, ..., 0]    // size MAX_LEN + 1  
    totalWords := 0  
    totalBytes := 0  
  
    OPEN wordFile  
    FOR EACH line in file:  
        cleanedLength := cleanLength(line)  
        IF cleanedLength is between 2 and MAX_LEN:  
            countPerLen[cleanedLength] += 1  
            totalWords += 1  
            totalBytes += cleanedLength
```

`cleanLength` is a helper that returns what the length would be if we accepted this line (after removing whitespace and rejecting non-letters). More on it below.

Allocate arrays at exactly the right size

```
wordData := new byte[totalBytes]  
wordOffset := new int[totalWords + 1]    // +1 for the end sentinel  
  
indicesByLength := empty 2D array of size MAX_LEN+1  
FOR L := 0 TO MAX_LEN:  
    indicesByLength[L] := new int[countPerLen[L]]  
  
// Per-length write cursor (how many words of length L have we placed?)  
lenCursor := [0, 0, 0, ..., 0]
```

Pass 2: Actually fill the arrays

```

writeByte := 0      // next empty spot in wordData
writeWord := 0      // next empty spot in wordOffset

OPEN wordFile again
FOR EACH line in file:
    len := fillBytes(line, wordData, writeByte)
    // fillBytes writes the cleaned lowercase letters into wordData
    // starting at writeByte, and returns how many bytes it wrote
    // (or -1 if it rejected the line)

    IF len == -1:
        CONTINUE    // skip rejected lines

    // Record this word's position
    wordOffset[writeWord] := writeByte

    // Add this word to its length bucket
    indicesByLength[len][ lenCursor[len] ] := writeWord
    lenCursor[len] += 1

    // Advance the write positions
    writeByte += len
    writeWord += 1

// Sentinel at the end: wordOffset[last+1] = total size of wordData
// This lets us compute the LAST word's length as offset[N] - offset[N-1]
wordOffset[writeWord] := writeByte

```

Precompute the first-guess cache

Why: The best first letter for a blank 5-letter word is always the same — it doesn't depend on the hidden word, just on what 5-letter dictionary words exist. So compute it once here instead of during every game.

```

firstGuess := new char[MAX_LEN + 1]    // all zeros
seenLocal  := new boolean[26]          // reusable "have we counted this letter
                                              // in the current word yet?" tracker

FOR L := 0 TO MAX_LEN:
    bucket := indicesByLength[L]
    IF bucket is empty:
        CONTINUE    // leave firstGuess[L] = 0, handled at game time

    // Count how many length-L words contain each letter
    counts := [0, 0, ..., 0]    // 26 zeros

    FOR EACH wordIdx in bucket:
        start := wordOffset[wordIdx]

        // Reset the "seen" tracker before looking at this word.
        // This ensures each letter is counted at most once per word.
        FOR i := 0 TO 25:
            seenLocal[i] := false

        FOR i := 0 TO L - 1:
            letter := wordData[start + i]
            b := letter - 'a'    // 0 to 25
            IF NOT seenLocal[b]:    // haven't counted this letter yet
                seenLocal[b] := true    // mark as counted
                counts[b] += 1

        // Best opening letter = highest count, ties broken by FREQ_ORDER
        best := 0
        bestCount := -1
        FOR EACH letter c in FREQ_ORDER:
            IF counts[c - 'a'] > bestCount:
                bestCount := counts[c - 'a']
                best := c

        firstGuess[L] := best

```

Why count each letter only once per word: We want to know "how many *words* contain

this letter," not "how many *total times* does this letter appear." A word like `banana` has three `a`s, but it should contribute 1 to the count for `a`, not 3. The `seenLocal[]` array is our memory of "which letters have I already counted from this word?" We reset it for each new word so counts don't leak across words.

Set up game-time buffers

```
guessed := new boolean[26]    // all false
seen := new boolean[26]      // reusable tracker for guess() function

// Pre-allocate candidates big enough to hold the biggest length bucket
maxBucket := 0
FOR L := 0 TO MAX_LEN:
    maxBucket := max(maxBucket, indicesByLength[L].length)

candidates := new int[max(maxBucket, 1)]
candCount := 0
firstGuessOfWord := false
```

Helper: cleanLength

What: Measures what the word's length would be after cleaning, without actually doing the cleaning. Returns -1 if the line should be rejected.

```
FUNCTION cleanLength(line) -> int:
    len := 0
    FOR i := 0 TO line.length - 1:
        c := line[i]
        IF c is whitespace (\r, \n, space, tab):
            CONTINUE // skip it
        IF c is NOT a letter (a-z or A-Z):
            RETURN -1 // reject the whole line
        len += 1
    RETURN len
```

Helper: fillBytes

What: Actually writes the cleaned lowercase letters into our byte array. Must use the same acceptance rules as `cleanLength`!

```
FUNCTION fillBytes(line, dest, destStart) -> int:
    k := 0
    FOR i := 0 TO line.length - 1:
        c := line[i]
        IF c is whitespace:
            CONTINUE
        IF c is uppercase (A-Z):
            c := lowercase(c)
        ELSE IF c is NOT lowercase (a-z):
            RETURN -1 // reject

        // Don't write past MAX_LEN even if we're going to reject
        // (otherwise we'd scribble into memory we don't own)
        IF k >= MAX_LEN:
            RETURN -1

        dest[destStart + k] := c as byte
        k += 1

    IF k < 2:
        RETURN -1 // we drop 1-letter words

    RETURN k
```

⚠ **Critical warning:** `cleanLength` and `fillBytes` must accept/reject the exact same lines. If they disagree, preprocessing will crash. The shared rules: - Skip whitespace (`\r`, `\n`, space, tab) - Reject if any other non-letter appears - Accept only if final length is 2 to `MAX_LEN`

Part 3: Making a Guess

What the evaluator calls: `guess(currentWord, isNewWord)` — and we return a letter to guess.
- `currentWord` is the board — something like " a _ _ e" with spaces for blanks - `isNewWord` is `true` only on the very first call for a fresh hidden word

Step 1: If it's a new word, reset

```
FUNCTION guess(currentWord, isNewWord) -> char:
    L := currentWord.length

    IF isNewWord:
        // Clear the list of letters we've guessed
        FOR i := 0 TO 25:
            guessed[i] := false

        // Seed candidates with all words of the right length
        IF L is between 0 and MAX_LEN:
            source := indicesByLength[L]
            candCount := source.length
            COPY source into candidates // use System.arraycopy in Java
        ELSE:
            candCount := 0 // weird length, will use fallback

        firstGuessOfWord := true
```

Step 2: First-guess shortcut

Why: On the very first guess of a word, we already precomputed the answer. Skip the scan.

```
IF firstGuessOfWord AND firstGuess[L] exists:
    best := firstGuess[L]
    IF we haven't guessed `best` yet:
        mark `best` as guessed
        firstGuessOfWord := false
    RETURN best

firstGuessOfWord := false
```

Step 3: Normal guess — count letters across candidates

The core idea: For each unguessed letter, count how many candidate words contain it *at least once*. The letter in the most words is our best guess, because it gives us the most information no matter how the guess turns out.

```
// Clear our counter
FOR i := 0 TO 25:
    letterCounts[i] := 0

// For each candidate word, count which letters it contains
FOR w := 0 TO candCount - 1:
    wordIdx := candidates[w]
    start := wordOffset[wordIdx]
    end := wordOffset[wordIdx + 1]

    // Reset the "seen" tracker before looking at this word, so each
    // letter is counted at most once PER WORD (word-containment, not
    // raw frequency: "banana" contributes 1 to 'a', not 3).
    FOR i := 0 TO 25:
        seen[i] := false

    FOR i := start TO end - 1:
        letter := wordData[i]
        b := letter - 'a'
        IF NOT seen[b]:
            seen[b] := true
            letterCounts[b] += 1
```

Step 4: Pick the winner

```
// Iterate in FREQ_ORDER so ties break by overall letter frequency
best := 0
bestCount := -1
FOR EACH letter c in FREQ_ORDER:
    b := c - 'a'
    IF guessed[b]:
        CONTINUE // skip letters we already guessed
    IF letterCounts[b] > bestCount:
        bestCount := letterCounts[b]
    best := c
```

Step 5: Fallback if nothing good was found

Why this can happen: If the hidden word isn't in our dictionary, candidates will eventually empty out or stop containing any unguessed letters. We need a safety net.

```
IF best == 0 OR bestCount == 0:
    // No candidates left (or all remaining letters had 0 count)
    // Just pick the next untried letter by general frequency
    FOR EACH letter c in FREQ_ORDER:
        IF NOT guessed[c - 'a']:
            best := c
            BREAK

mark 'best' as guessed
RETURN best
```

Part 4: Processing Feedback

What the evaluator calls: `feedback(isCorrectGuess, currentWord)` after we make a guess. - `isCorrectGuess` tells us if the letter was in the hidden word - `currentWord` is the updated board

Our job: Narrow down the candidate list to only words still possible.

Quirk: We actually ignore `isCorrectGuess`! Everything we need is in `currentWord` plus our own `guessed` list. Less parameters to worry about = fewer bugs.

```
FUNCTION feedback(isCorrectGuess, currentWord):
    L := currentWord.length
    write := 0 // where to put the next surviving candidate

    FOR read := 0 TO candCount - 1:
        wordIdx := candidates[read]
        start := wordOffset[wordIdx]
        end := wordOffset[wordIdx + 1]

        // Defensive length check
        IF (end - start) != L:
            CONTINUE

        IF consistent(start, currentWord, L):
            candidates[write] := wordIdx
            write += 1
        // else: just drop it (don't copy it forward)

    candCount := write
```

The "two-pointer" trick: We walk through candidates with `read` and write survivors back at `write`. Since `write <= read` always, we can safely overwrite as we go. No new array needed.

Helper: consistent

This is the most important function in the whole program. It answers: "Is this candidate

word still possible as the hidden word?"

There are only two ways a candidate can be eliminated: 1. A revealed position doesn't match the candidate 2. A blank position has a letter in the candidate that we already guessed

Why rule 2? Here's the key insight: if we guessed a letter and it was in the hidden word, the evaluator reveals *every* occurrence of it at once. So if a position is still blank, we know the hidden word does NOT have any of our guessed letters there. If our candidate has a guessed letter in that blank spot, it can't be right.

```
FUNCTION consistent(start, currentWord, L) -> boolean:
  FOR i := 0 TO L - 1:
    candidateLetter := wordData[start + i]
    patternChar := currentWord[i]

    IF patternChar is a letter (not blank):
      // Revealed spot: must match exactly
      IF candidateLetter != patternChar:
        RETURN false
    ELSE:
      // Blank spot: candidate's letter here must NOT be one we've guessed
      IF guessed[candidateLetter - 'a']:
        RETURN false

  RETURN true
```

Walk through an example: Hidden word is `BAT`, pattern shown is `_A_`, we've guessed {A, T}. Check candidate `HAT`: - Position 0: pattern blank. Candidate has 'H'. Have we guessed 'H'? No. OK. - Position 1: pattern shows 'A'. Candidate has 'A'. Match. OK. - Position 2: pattern blank. Candidate has 'T'. Have we guessed 'T'? **Yes**. Reject!

Why reject? Because if the hidden word had 'T', we would have seen it revealed at position 2. The pattern still shows blank there, so the hidden word has no 'T'. But HAT has 'T'. So HAT isn't the hidden word.

Part 5: The Flow, All Together

Here's what a complete game looks like from our program's perspective:

```
The evaluator calls our constructor ONCE:
HangmanPlayer(wordFile)
-> reads dictionary, builds all our data structures

THEN for each hidden word the evaluator tests:

The evaluator calls guess() with isNewWord = true
-> we reset guessed[], refill candidates[] with all words of that length
-> we return firstGuess[L] (our precomputed best opener)

The evaluator calls feedback(result, updatedPattern)
-> we filter candidates down to ones matching the updated pattern

LOOP until word is solved or 6 wrong guesses:
  The evaluator calls guess() with isNewWord = false
  -> we scan candidates, count letter-in-word, pick the winner
  -> we return our guess

  The evaluator calls feedback(result, updatedPattern)
  -> we filter candidates again
```

Part 6: What People Get Wrong

If someone implements this and gets poor results, check these in order:

1. **Did they reset state on `isNewWord`?** The evaluator reuses the same `HangmanPlayer` object. If `guessed[]` and `candidates[]` carry over from the previous word, accuracy tanks.

2. **Did they count each letter once per word, not per occurrence?** If "banana" contributes 3 to the count for 'a' instead of 1, the algorithm picks bad letters. The `seen[]` array is what prevents this — make sure it gets reset before each word.
3. **Does `consistent()` enforce BOTH rules?** Many people only check the "revealed positions match" rule and forget "blank positions must not contain guessed letters." Without both, candidates stop shrinking correctly.
4. **Did `cleanLength` and `fillBytes` stay in sync?** If they disagree on whether to accept a word, preprocessing crashes.
5. **Did they use `Scanner` instead of `BufferedReader`?** `Scanner` works but eats huge memory on this dictionary. The assignment warns about this.

Keep this document next to your code while you write it — every function in your implementation should match a section here.